LANGCHAIN v0.3 TRAINING

Building Context-Aware LLM Applications

From Zero to Production with LCEL, RAG, and Agents.

The Problem: Standalone LLMs

Stale Knowledge

Training data cutoffs mean models don't know about events from yesterday.

No Privacy

They don't know your company's HR policies, SQL data, or emails.

No Action

They only output text strings; they cannot execute API calls natively.

The Solution: LangChain

A framework to connect LLMs to your world.

- **Context Awareness:** Connects LLMs to sources of data (RAG).
- **Reasoning:** Relies on LLMs to decide *how* to answer (Agents).
- **Orchestration:** Chains disparate components into a single pipeline.

The Architecture: LCEL

LangChain Expression Language

A declarative, Linux-style syntax for chaining components.

```
chain = Prompt | Model | Parser
```

Why? It standardizes streaming, async support, and debugging across all chains.

The Runnable Protocol

Every component in LangChain implements standard methods:

- `.invoke()` - Call it.
- `.stream()` - Stream it.
- `.batch()` - Run parallel inputs.

1. Prompts: The Theory

Structured Inputs

Modern models don't just take a string; they take a list of **Messages**.

- **System Message:** Instructions ("You are a helpful assistant").
- **User Message:** The input ("Hello").
- **AI Message:** The model's response.

Templates & Variables

Hardcoding strings is bad practice. We use **Templates** with dynamic slots (`{variable}`) to ensure consistency and reusability at scale.

1. Prompts: The Code

Using `ChatPromptTemplate` with `MessagesPlaceholder`.

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

# Define a robust prompt structure
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert in {topic}."),

    # Critical for Memory: Inject past history here
    MessagesPlaceholder(variable_name="history"),

    ("user", "{question}")
])

# Usage
val = prompt.invoke({
    "topic": "Physics",
    "history": [],
    "question": "Explain gravity."
})
```

2. Models: The Theory

ChatModels vs. LLMs

LLM (Legacy): String In → String Out.

ChatModel (Standard): Messages In → Message Out.

Even for completion tasks, ChatModels are now the standard because they support system instructions and tool calling.

Temp

Controls randomness. 0.0 for facts, 0.8 for creativity.

Swap

Standard interfaces allow swapping OpenAI for Gemini instantly.

2. Models: The Code

Best practice v0.3: Use the factory `init_chat_model`.

```python
from langchain.chat_models import init_chat_model

# 1. Initialize (Model Agnostic)
model = init_chat_model(
    "gpt-4o",
    model_provider="openai",
    temperature=0
)

# 2. Invoke (Blocking)
response = model.invoke("Hello!")

# 3. Stream (Real-time)
for chunk in model.stream("Write a poem."):
    print(chunk.content, end="")
```

3. Runnable Interface: The Theory

The "Runnable" is the atomic unit of LangChain. If it's a Runnable, it can be piped.

.stream()

Iterates output chunks. Vital for UI feedback (typing effect).

.batch()

Processes a list of inputs in parallel using thread pools.

.ainvoke()

Async version. Essential for building high-concurrency web servers.

3. Runnable Interface: The Code

Using `RunnableParallel` to fetch data and process inputs simultaneously.

```
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# A typical RAG setup
chain = (
    RunnableParallel({
        "context": retriever,        # Fetch docs
        "question": RunnablePassthrough() # Pass input as-is
    })
    | prompt
    | model
    | parser
)

# The input "What is AI?" goes to BOTH retriever and question keys
chain.invoke("What is AI?")
```

4. Chains: The Theory

Legacy vs. Modern

Legacy (v0.1): Used classes like `LLMChain`. These were "black boxes" that were hard to debug.

Modern (v0.3): Chains are just sequences of Runnables connected by pipes (`|`).

Visibility

Modern chains expose every step. You can inspect the output of the prompt before it hits the model.

4. Chains: The Code

Composing and Debugging Chains.

```
from langchain.globals import set_debug

# Turn on verbose logging to see every step in the terminal
set_debug(True)

# Composition
chain = prompt | model | StrOutputParser()

# Execution logs will show:
# [chain/start] Entering Chain run...
# [llm/start] Entering LLM run...
val = chain.invoke({"topic": "Space"})
```

Output Parsers

LLMs output objects. Parsers turn them into data.

StrOutputParser

```
"Hello world"
```

Extracts the string content from the AIMessage. Removes metadata.

JsonOutputParser

```
{"key": "value"}
```

Forces the LLM to output valid JSON and converts it to a Python Dictionary.

RAG Part 1: Ingestion

1. Load & Split

Use `PyPDFLoader` to read files. Use `RecursiveCharacterTextSplitter` to break them into 500-token chunks.

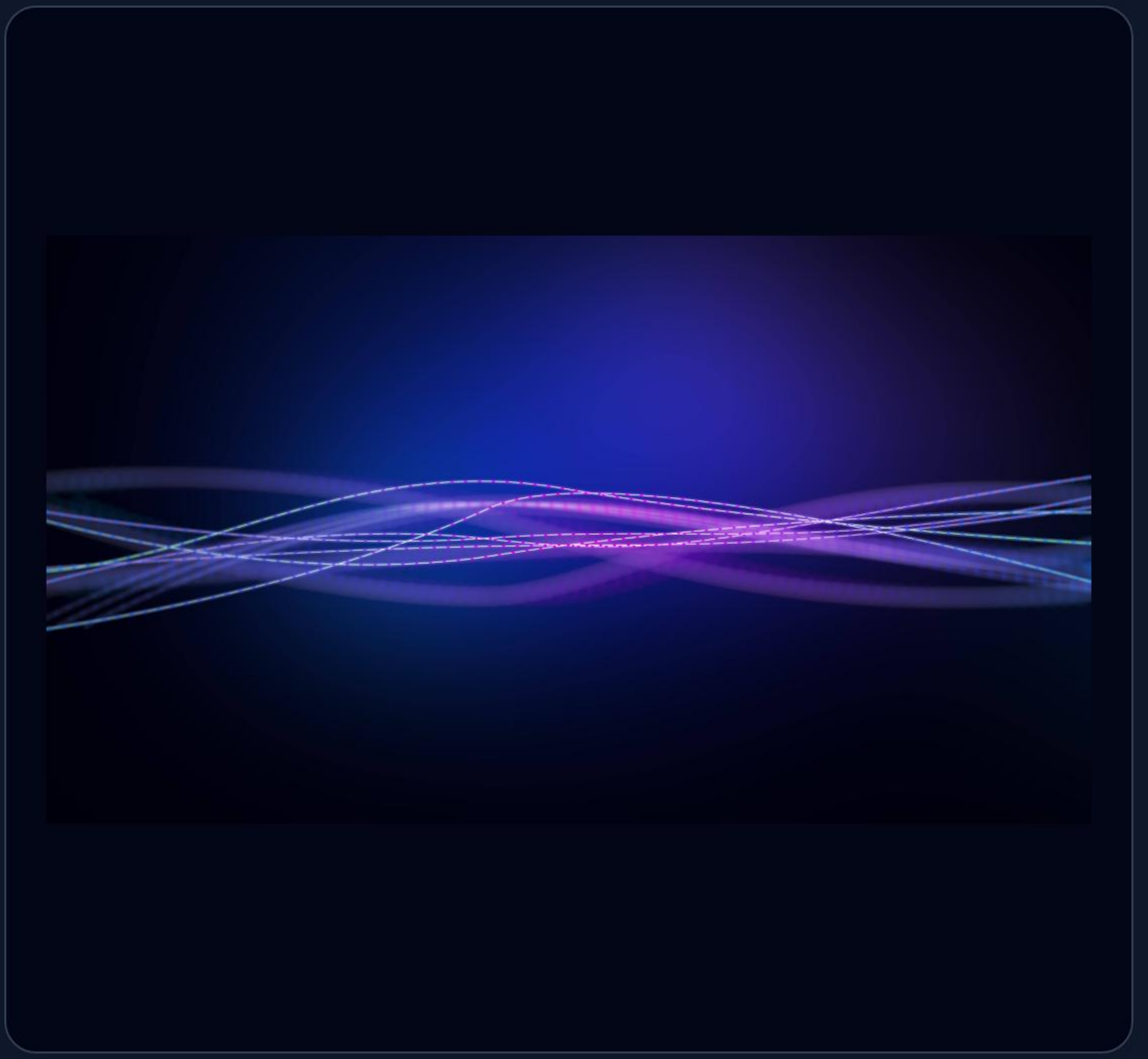
2. Embed & Store

Convert text chunks into Vectors (numbers) using an embedding model. Save them in a VectorStore (ChromaDB).

Goal: Create a searchable index of your private data.

RAG Part 2: Retrieval Flow

1. **User asks a question.**
2. **Retrieve:** System searches DB for top 3 similar chunks.
3. **Augment:** System pastes chunks into the Prompt Context.
4. **Generate:** LLM answers using the Context.



5. State: The Theory

The Stateless Problem

LLMs are like a goldfish; they have no memory of the past. Every API call is independent.

State Management

"State" is the infrastructure we build to persist context (messages, variables) across turns. It must be stored in a database (Redis/SQL).

LangGraph

For complex state, we use **Schemas** (TypedDict) to strictly define what data passes between steps in an agent.

5. State: The Code

Defining State Schemas for Agents.

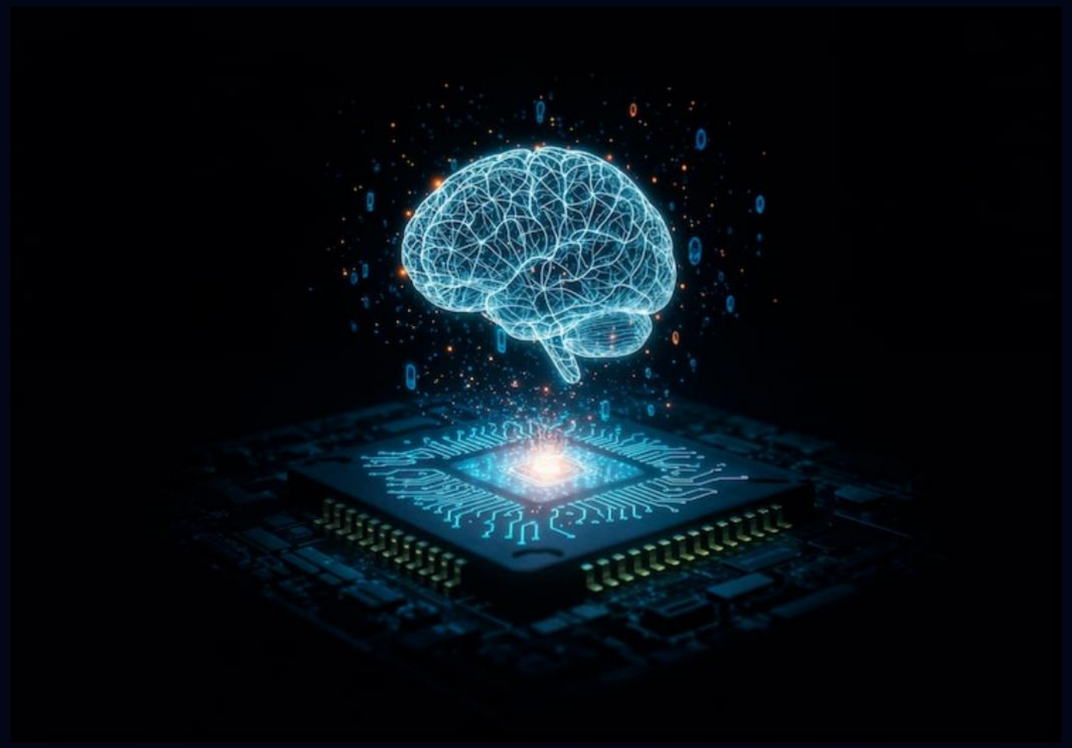
```python
from typing import TypedDict, Annotated
from langgraph.graph import add_messages

# Define the shape of our State
class AgentState(TypedDict):
    # 'add_messages' reducer appends new msgs instead of overwriting
    messages: Annotated[list, add_messages]
    user_id: str

# Nodes receive State and return Updates
def chatbot(state: AgentState):
    return {"messages": [response_msg]}
```

6. Memory: The Theory

Thread-Scoped History

Memory is a specific implementation of State for **Chat History**.

It acts as a wrapper around your Chain. It intercepts the input, looks up the `session_id` in a database, and injects past messages into the prompt.



6. Memory: The Code

Using `RunnableWithMessageHistory`.

```
from langchain_core.runnables.history import RunnableWithMessageHistory

# Wrap your chain
chain_with_history = RunnableWithMessageHistory(
    runnable=chain,
    get_session_history=get_history_from_db,  # Function to fetch from Redis
    input_messages_key="question",
    history_messages_key="history"
)

# Invoke with session ID
chain_with_history.invoke(
    {"question": "Hi, I'm Bob"},
    config={"configurable": {"session_id": "user_1"}}
)
```

Agents & Ecosystem

Agents

LLMs that use **Tools**. The model reasons: "I need to check the weather," calls the Weather API, and then answers the user.

The Ecosystem

- **LangSmith:** Debugging & Tracing (Critical).
- **LangServe:** One-line API deployment.
- **LangGraph:** Multi-actor stateful agents.

Image Sources

https://png.pngtree.com/background/20250804/original/pngtree-futuristic-network-of-interconnected-nodes-linked-by-glowing-blue-lines-set-picture-image_16855659.jpg

Source: pngtree.com

https://static.vecteezy.com/system/resources/thumbnails/035/617/169/original/abstract-flowing-gradient-fluid-waves-motion-of-digital-data-flow-colorful-wavy-background-in-bright-purple-blue-color-on-dark-backdrop-digital-neon-motion-wallpaper-4k-60fps-seamless-animation-video.jpg

Source: www.vecteezy.com

https://img.freepik.com/free-photo/artificial-intelligence-neural-network-visualization_23-2151977482.jpg?semt=ais_hybrid&w=740&q=80

Source: www.freepik.com